Sprint1

Team: Trinix

Additions:

- Added sys_getsystemcallscount system call, which returns the call count of each systemcall by returning an array of size (count of systemcalls +1)
- This is done so that when we return, each index in the array corresponds to a system call of such number (index 1 for system call 1...). The method keeps count of each system call by having a global array in syscall.c and increment the value at index num inside the main of syscall.c.
- This ensures that we are keeping correct track of the number of calls, as all system calls must pass through syscall.c.
- Added 3 user programs, one for general purpose, displaying all invocation counts of all system calls when called, called scount. And 2 test user programs named scountTest1 and scountTest2 which invoke some systemcalls and compare the values I get from getsystemcallscount before and after I call these system calls. This is done to assess whether my implementation is keeping track accurately, or it fails to do so.

scountTest1 being invoked

and returning expected, correct results.

```
$ scountTest1
Syscall counts before and after:
fork:diff = 1
exit:diff = 1
wait:diff = 1
pipe:diff = 0
read:diff = 0
kill:diff = 0
exec:diff = 0
fstat:diff = 0
chdir:diff = 0
dup:diff = 0
getpid:diff = 1
sbrk:diff = 0
sleep:diff = 0
uptime:diff = 0
open:diff = 0
write:diff = 0
mknod:diff = 0
unlink:diff = 0
link:diff = 0
mkdir:diff = 0
close:diff = 0
getsystemcallscount:diff = 1
```

scountTest2 being invoked, testing different system calls compared to scountTest1, and returning correct, expected results.

```
$ scountTest2
NOTE: we have st
Syscall counts before and after:
fork:diff = 0
exit:diff = 0
wait:diff = 0
pipe:diff = 0
read:diff = 1
kill:diff = 0
exec:diff = 0
fstat:diff = 0
chdir:diff = 0
dup:diff = 0
getpid:diff = 0
sbrk:diff = 0
sleep:diff = 1
uptime:diff = 0
open:diff = 1
write:diff = 1
mknod:diff = 0
unlink:diff = 0
link:diff = 0
mkdir:diff = 0
close:diff = 1
getsystemcallscount:diff = 1
$ scount
```

The general use scount systemcall, this user program is intended to be the user program that is used by the user to check how many times each system calls have been invoked thus far(till calling scount).

Documentation:

The following files were altered/added to the original xv6 program:

1. user.h
2. syscall.h
3. syscall.c
4. scount.c
5. scountTest1.c
6. scountTest2.c
7. Makefile
8. usys.S
9. sysproc.c

The following will explain what was done to each changed/added element:

For 1:

- Added getsystemcallscount declaration

For 2:

- Defined SYSCALLSMETHODS and made it equal to number of system calls +1
- Defined SYS_getsystemcallscount with value 22(1+max old value)

For 3:

- Added global array counts of size SYSCALLSMETHODS to keep track of array invocation count inside the main of syscall.c
- Added extern of our new system call and added it to syscalls array.

For 4:

- User program that calls getsystemcallscount and print values of each index and the corresponding name of the systemcall

For 5:

- User program that is intended to test out system call by comparing the invocation count before and after invoking some systemcalls

For 6:

- Same as 5 but using different systemcalls

For 7:

- Added our userprograms into Uprogs and Extra(as told by the geeksforgeeks tutorial, to make the compiler treat our program as user programs)

For 8:

- Added SYSCALL(getsystemcallscount) (to allow for user program to call kernel program)

For 9:

- Added the actual implementation of our systemcall by using each of argptr and copyout to copy and modify the user-passed array inside our function while making sure no errors occur while doing so